Chapter 11

Numbers and time

count exactly, measure approximately, keep time carefully

Python has five kinds of number. Picking the wrong one gives answers that are **almost** right, which is worse than an error because nothing tells you:

type	exact?	size	use it for
`int`	**yes**	unlimited: grows as needed	counting, indexes, ids, money stored as pence
`float`	**no**: nearest binary fraction	64 bits, about 15-17 significant digits	measurements, science, graphics
`Decimal`	**yes** for base-10 digits	you choose the precision	money, anything a person types in decimal
`Fraction`	**yes**	unlimited	exact ratios, probability, maths
`complex`	as two floats	two 64-bit floats	signals, geometry, `(-1) ** 0.5`

```
1 from decimal import Decimal
2 from fractions import Fraction
3
4 print(2 ** 100)                                  # ints never overflow
5 print(0.1 + 0.2, 0.1 + 0.2 == 0.3)               # floats are approximations
6 print(Decimal("0.1") + Decimal("0.2"), Fraction(1, 3) + Fraction(1, 6))
7 print(7 / 2, 7 // 2, type(2 ** 0.5).__name__, (-1) ** 0.5)
```

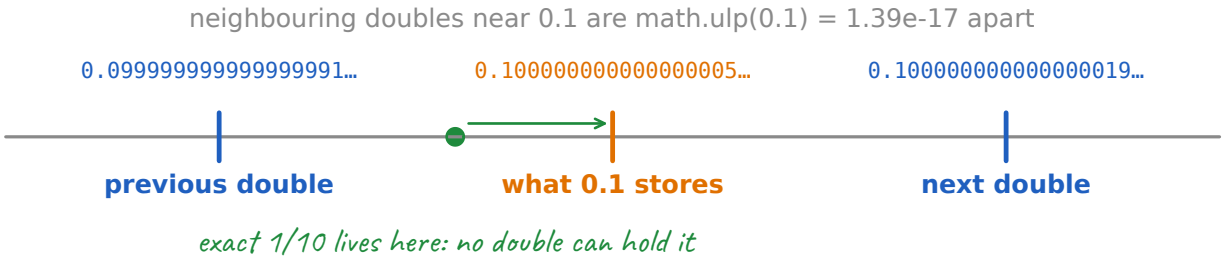
```
1267650600228229401496703205376
0.30000000000000004 False
0.3 1/2
3.5 3 float (6.123233995736766e-17+1j)
```

line 5: the famous one: next page explains it

Time is worse. Clocks get corrected, countries move their clocks twice a year, and "3 pm" means nothing without a place. The second half of the chapter gives you the three rules that avoid nearly every time bug.

Why 0.1 + 0.2 isn't 0.3

A float stores `sign × 1.fraction × 2^exponent` with 52 bits of fraction. Only numbers that are sums of powers of two fit exactly. One tenth in binary is `0.0001100110011001100…` forever, just as 1/3 is `0.333…` in decimal, so Python stores the **nearest** double instead:




```
1 import math
2 from decimal import Decimal
3
4 print(Decimal(0.1))                          # the exact value actually stored
5 print((0.1).as_integer_ratio())              # = that value as a fraction over 2**55
6 print(0.1 + 0.2)                            # two rounding errors, then a third
7 print(math.isclose(0.1 + 0.2, 0.3), abs((0.1 + 0.2) - 0.3) < 1e-9)
8 print(sum([0.1] * 10), math.fsum([0.1] * 10))
9 print(math.ulp(1.0), math.ulp(1e16), 1e16 + 1 == 1e16)
```

```
0.1000000000000000055511151231257827021181583404541015625
(3602879701896397, 36028797018963968)
0.30000000000000004
True True
1.0 1.0
2.220446049250313e-16 2.0 True
```

line 7: compare with a tolerance

line 8: fsum tracks the lost bits

line 9: big floats have big gaps

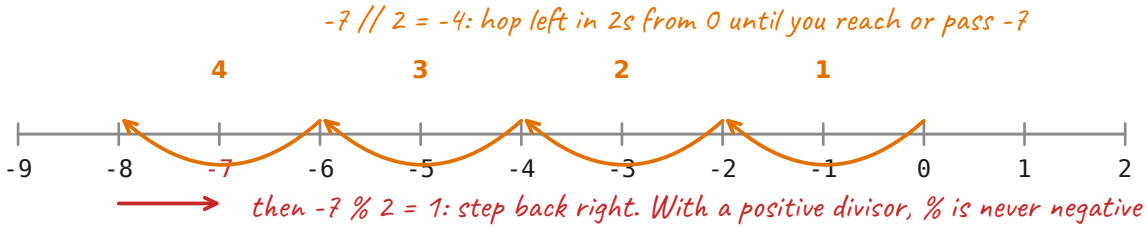
trap	what happens	do this instead
`a == b` on results	tiny differences make equal values unequal	`math.isclose(a, b)`
adding many floats	errors pile up	`math.fsum`, or Decimal / int pence
huge + tiny	the tiny one vanishes (`1e16 + 1`)	rescale, or sort small-first
`float("nan")`	`nan != nan`, and it spreads through maths	`math.isnan(x)`
money in floats	`0.1 + 0.2` pence errors in totals	`Decimal("0.10")` or int pence

IN OTHER LANGUAGES

This is IEEE 754, built into the CPU. C `double`, Java `double`, JavaScript `Number` (its **only** number type), Rust `f64` all print `0.30000000000000004`. It is not a Python bug.

Integers, division and rounding

Python ints grow as large as memory allows, so there is no overflow. `/` always gives a float; `//` gives the **floor** (rounds towards minus infinity), and `%` is whatever is left over:




```
1 import math
2
3 print(2 ** 200 % 1000, (2 ** 200).bit_length())
4 print(7 // 2, -7 // 2, 7 % 3, -7 % 3, divmod(-7, 2))
5 print(int(-3.7), math.floor(-3.7), math.trunc(-3.7), math.ceil(-3.7))
6 print(round(2.5), round(3.5), round(0.125, 2), round(2.675, 2))
7 print(pow(3, 200, 1000), math.gcd(12, 18), math.lcm(4, 6), math.isqrt(99))
8 print(math.comb(5, 2), math.perm(5, 2), math.factorial(20))
```

```
376 201
3 -4 1 2 (-4, 1)
-3 -4 -3 -3
2 4 0.12 2.67
1 6 12 9
10 20 2432902008176640000
```

line 4: floor: -3.5 becomes -4

line 6: half to even; 2.675 is really 2.67499...

line 7: modular power: fast, no huge number

WHY

Flooring keeps `a == (a // b) * b + a % b` true **and** keeps `%` the same sign as the divisor, so `(i - 1) % n` wraps round a list cleanly. C, Java and JavaScript truncate towards zero instead: `-7 / 2` is `-3` and `-7 % 2` is `-1` there. Port maths code carefully.

you want	use	2.5 →	-2.5 →
round to nearest, ties to even (less bias)	`round(x)`	2	-2
round half up (school rounding)	`Decimal.quantize(..., ROUND_HALF_UP)`	3	-3
always down / always up	`math.floor` / `math.ceil`	2 / 3	-3 / -2
chop the fraction off	`int(x)` or `math.trunc(x)`	2	-2

decimal and fractions: exact arithmetic

`Decimal` does arithmetic in base 10, the way you do it on paper, so `0.10` stays exactly `0.10`. It's slower than float, and that never matters for money:

float

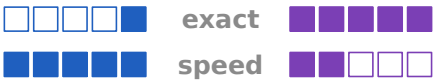
```
prices = [0.10] * 3
total = sum(prices)
print(total, total == 0.30)
print(f"{total:.2f}")
```

```
0.30000000000000004 False
0.30
```

Decimal

```
from decimal import Decimal
prices = [Decimal("0.10")] * 3
total = sum(prices)
print(total, total == Decimal("0.30"))
```

```
0.30 True
```




```
1  from decimal import Decimal, ROUND_HALF_UP, localcontext
2
3  print(Decimal(0.1) == Decimal("0.1"))         # made from a float: inherits its error
4  price = Decimal("19.99")
5  vat = (price * Decimal("0.20")).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
6  print(vat, price + vat)
7  print(Decimal("2.675").quantize(Decimal("0.01")), Decimal(1) / Decimal(7))
8  with localcontext() as ctx:
9      ctx.prec = 50                             # digits of precision, for this block only
10     print(Decimal(1) / Decimal(7))
```

```
False
4.00 23.99
2.68 0.14285714285714285714285714286
0.14285714285714285714285714285714285714285714285714
```

line 3: always build from strings

line 5: quantize = round to these places

Fraction: exact ratios

```
1 import math
2 from fractions import Fraction
3
4 x = Fraction(1, 3) + Fraction(1, 6)
5 print(x, x.numerator, x.denominator, float(x))
6 print(Fraction("0.75"), Fraction(0.1) == Fraction(1, 10))
7 print(Fraction(math.pi).limit_denominator(1000))
8 print(sum(Fraction(1, n) for n in range(1, 11)))
```

```
1/2 1 2 0.5
3/4 False
355/113
7381/2520
```

← best fraction with a small denominator

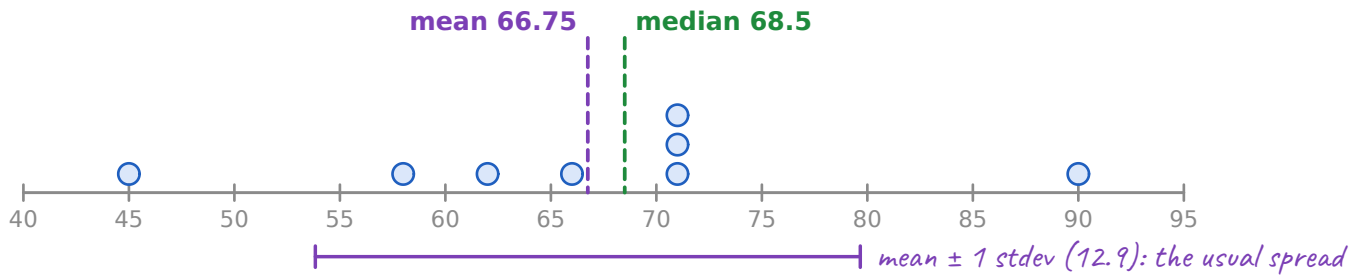
1 + 1/2 + … + 1/10, exactly

RULE OF THUMB

Money: `Decimal` built from strings, or whole pence in an `int`. Measurements: `float` and compare with `math.isclose`. Exact ratios: `Fraction`. Never `Decimal(some_float)`: the error is already inside the float.

math and statistics

`statistics` covers what a first statistics module teaches, with careful numerics. Here are eight exam marks drawn as dots:




```
1 import statistics as st
2
3 marks = [62, 71, 71, 58, 90, 45, 71, 66]
4 print(st.mean(marks), st.median(marks), st.mode(marks))
5 print(round(st.stdev(marks), 2), round(st.pstdev(marks), 2))
6 print(st.quantiles(marks, n=4))                    # the three quartile cut points
7
8 hours, score = [2, 4, 5, 7, 9], [50, 62, 65, 74, 88]
9 fit = st.linear_regression(hours, score)
10 print(round(st.correlation(hours, score), 3), round(fit.slope, 2), round(fit.intercept, 2))
```

```
66.75 68.5 71
12.91 12.08
[59.0, 68.5, 71.0]
0.994 5.22 39.62
```

line 5: sample (n-1) vs population (n)

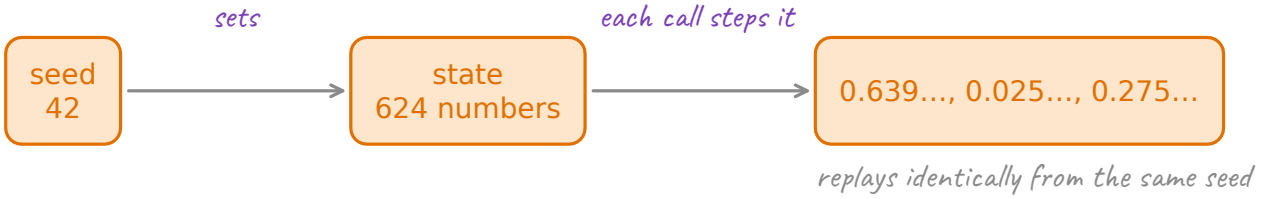
WHY

`stdev` divides by n − 1 because your data is a **sample**: its own mean sits closer to its points than the true mean would, so dividing by n underestimates the spread. Use `pstdev` only when you have the whole population (every student, not some).

math has...	for
`sqrt`, `isqrt`, `cbrt`, `exp`, `log`, `log2`, `log10`	roots and logs (`log(x, base)` too)
`sin`, `cos`, `atan2`, `radians`, `degrees`	trigonometry, in radians
`hypot`, `dist`, `prod`, `sumprod`	lengths, distances, products
`pi`, `tau`, `e`, `inf`, `nan`	constants
`isclose`, `isfinite`, `isnan`, `nextafter`, `ulp`	float housekeeping

random versus secrets

`random` is a **pseudo**-random generator: a deterministic formula (the Mersenne Twister) that turns a seed into a long sequence that only looks random. Same seed, same sequence:




```
1  import random
2
3  rng = random.Random(42)                       # your own generator, reproducible
4  print(rng.randint(1, 6), rng.choice("ABC"), round(rng.random(), 3))
5  again = random.Random(42)
6  print(again.randint(1, 6), again.choice("ABC"), round(again.random(), 3))
7  deck = list(range(10))
8  rng.shuffle(deck)
9  print(deck)
10 print(rng.sample(range(100), 3), rng.choices("HT", weights=[9, 1], k=8))
11 print(round(rng.gauss(0, 1), 3), round(rng.uniform(1.5, 2.5), 3))
```

```
6 A 0.025
6 A 0.025
[7, 6, 2, 9, 0, 5, 1, 8, 3, 4]
[4, 3, 11] ['H', 'H', 'H', 'H', 'H', 'H', 'H', 'H']
0.042 2.306
```

line 6: identical: same seed

line 10: sample: no repeats; choices: repeats, weighted

TRAP

Watch 624 outputs of `random` and you can rebuild its state and predict every later number. Fine for games, simulations and tests; useless against an attacker. Tokens, passwords, keys and anything with money riding on it use `secrets` (Ch 13), which reads the OS's cryptographic generator.

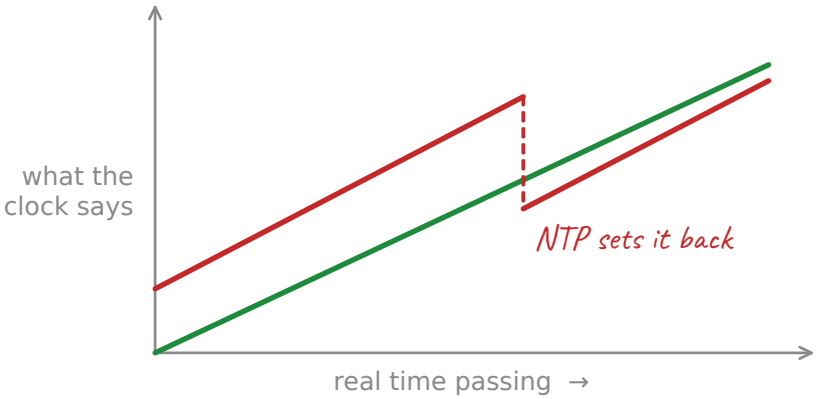
you need	use
a simulation, test data or game you can replay	`random.Random(seed)` (pass the rng in)
shuffle quiz questions	`random.shuffle`
a password-reset token or session id	`secrets.token_urlsafe(32)`
pick the winner of a paid draw	`secrets.choice(entries)`
millions of random numbers for NumPy	`np.random.default_rng(seed)` (Ch 18)

RULE OF THUMB

Pass a `random.Random` object into code that needs randomness instead of calling the global `random.randint`. Tests can then give it a fixed seed, or a fake, and get the same result every run. Practice 11.6 makes you do exactly this.

Three clocks, three jobs

A computer has more than one clock. Using the wrong one is how a 30-second timeout fires instantly, or a benchmark reports negative time:



time.time(): the wall clock. Seconds since 1970 UTC. Can jump either way (NTP sync, a user changing it, a VM resuming).

time.monotonic(), **time.perf_counter()**: never go backwards. Their zero means nothing; only differences do.

call	use it for	can jump?
`time.time()`	timestamps you store: when did this happen?	**yes**
`time.monotonic()`	timeouts, deadlines, retry back-off	**never back**
`time.perf_counter()`	measuring how long code takes (finest resolution)	**never back**
`time.process_time()`	CPU time used by this process (ignores sleeping)	**never back**
`time.sleep(s)`	pause the thread for at least s seconds	

```python
import time

print(time.get_clock_info("time").adjustable, time.get_clock_info("monotonic").adjustable)

deadline = time.monotonic() + 0.05            # 50 ms from now, immune to clock changes
polls = 0
while time.monotonic() < deadline:
    polls += 1
    time.sleep(0.01)
print(polls >= 3)
```

```
True False
True
```

line 3: True: can be set; False: can't

line 5: the deadline pattern

datetime: naive, aware, and the time zone

A `datetime` with no `tzinfo` is **naive**: "14:30 on 28 Sept", but where? One with a zone from `zoneinfo` is **aware**: a single instant everyone agrees on. Only aware times can be converted or compared across places:

```python
from datetime import date, datetime, timedelta, timezone
from zoneinfo import ZoneInfo

naive = datetime(2026, 9, 28, 14, 30)
aware = datetime(2026, 9, 28, 14, 30, tzinfo=ZoneInfo("Europe/London"))
print(naive.tzinfo, aware.utcoffset(), aware.tzname())
print(aware.astimezone(timezone.utc))
print(aware.astimezone(ZoneInfo("Asia/Tokyo")).isoformat())
print(date(2026, 12, 25) - date(2026, 9, 28), date(2026, 9, 28).strftime("%A %d %B"))
print(datetime.fromisoformat("2026-09-28T14:30:00+01:00") == aware)
```

```
None 1:00:00 BST
2026-09-28 13:30:00+00:00
2026-09-28T22:30:00+09:00
88 days, 0:00:00 Monday 28 September
True
```

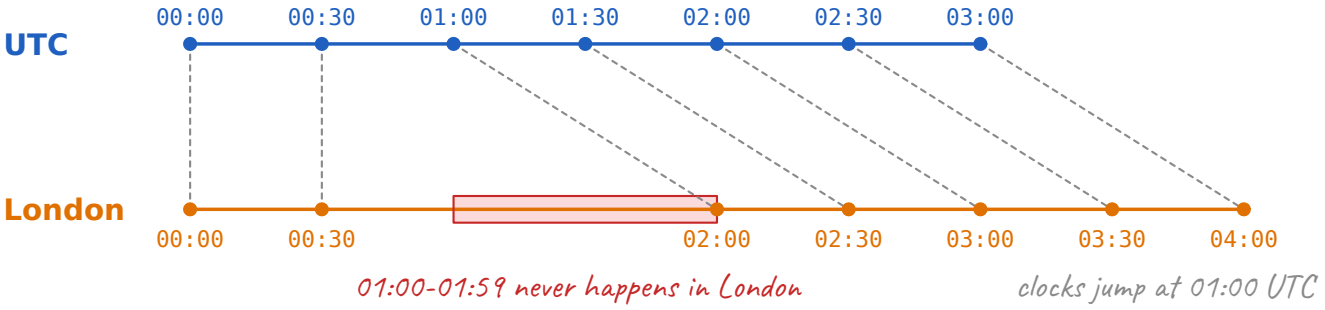
line 7: same instant, other zone

line 10: ISO 8601: the format to store and send

```python
print(naive < aware)
```

```
TypeError: can't compare offset-naive and offset-aware datetimes
```

Clock changes: the gap and the fold




```
1 london = ZoneInfo("Europe/London")
2 before = datetime(2026, 3, 28, 12, 0, tzinfo=london)
3 after = before + timedelta(days=1)            # wall-clock arithmetic within one zone
4 print(after, after - before)
5 print(after.astimezone(timezone.utc) - before.astimezone(timezone.utc))
6 gap = datetime(2026, 3, 29, 1, 30, tzinfo=london)     # never happened
7 print(gap.astimezone(timezone.utc).astimezone(london))
8 twice = datetime(2026, 10, 25, 1, 30, tzinfo=london)  # happened twice
9 print(twice.utcoffset(), twice.replace(fold=1).utcoffset())
```

```
2026-03-29 12:00:00+01:00 1 day, 0:00:00
23:00:00
2026-03-29 02:30:00+01:00
1:00:00 0:00:00
```

line 4: says 1 day…

line 5: …but only 23 hours passed

line 7: shifted out of the gap

line 9: fold picks which 01:30

RULE OF THUMB

1. Store and compute in **UTC**, as aware datetimes: `datetime.now(timezone.utc)` (never `utcnow()`, which is deprecated and naive). 2. Convert to local time only to show a person. 3. Send and store times as ISO 8601 strings with the offset. Repeating events ("every Monday at 9 local") are the one case where you compute in local time, then convert (practice 11.4).

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 11.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 11.1` shows other valid solutions and their trade-offs.

`11.1` build from a spec Split a bill to the penny
`11.2` fix the bug A float range that doesn't drift
`11.3` fix the bug Running mean and variance that stay accurate
`11.4` fix the bug A weekly meeting across clock changes
`11.5` learn an unseen module Calendar questions with an unseen module
`11.6` mini-project A seeded dice roller

PASSPORT STAMP ask the same question of every new language (Appendix B)

3. Typing

Numbers: `int` is unlimited; `float` is an IEEE 754 double (inexact); `Decimal` and `Fraction` are exact library types. `/` gives float, `//` floors. Booleans are ints (`True + True == 2`).

12. Stdlib and tooling

`math`, `statistics`, `random` (reproducible, not secure), `secrets`, `decimal`, `fractions`, `time` (three clocks), `datetime` + `zoneinfo`.

IN OTHER LANGUAGES

C and Java ints have fixed sizes and overflow (silently in C, wrapping in Java); Rust panics on overflow in debug builds. Java has `BigDecimal`, C# `decimal`, JavaScript `BigInt`. Every language has the same naive/aware time split: Java `LocalDateTime` vs `ZonedDateTime`, JS `Temporal.PlainDateTime` vs `Temporal.ZonedDateTime`.

DEEP DIVE (your vault)

Vault notes: 5.9 Numeric objects · 16.3 Descriptive location and dispersion · 16.11 Reductions, axes, and numerical stability · 16.12 Randomness, generators, and reproducibility · 16.21 Time data, ordering, and time zones · 19.22 Cryptographic randomness.